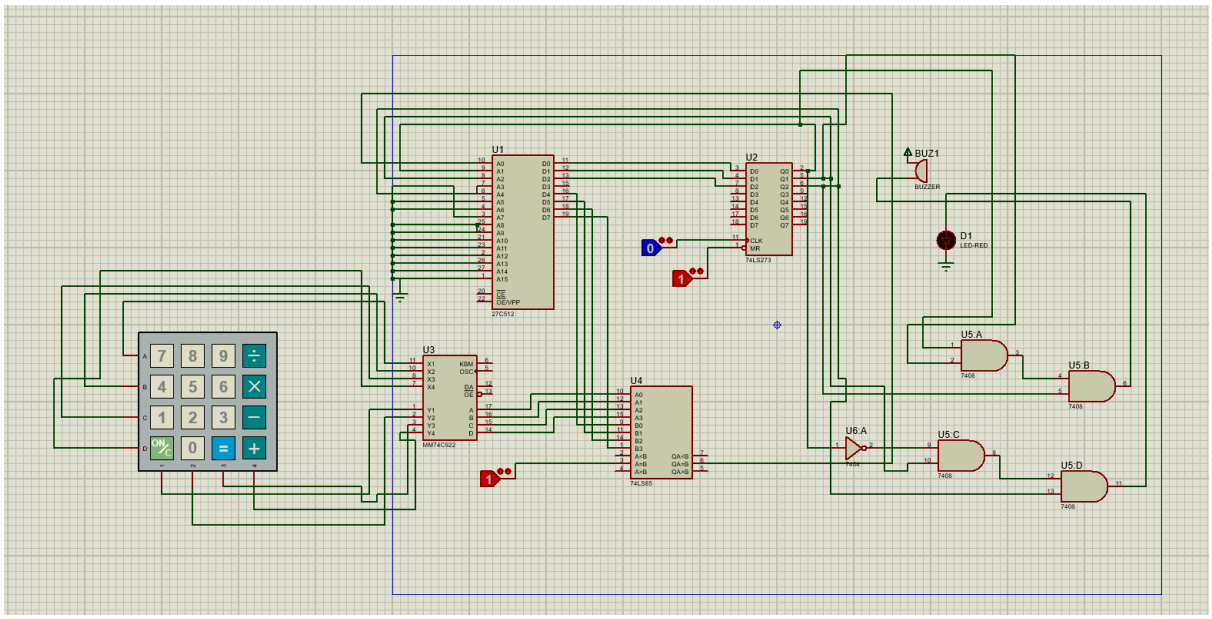
ROM-Based Digital Lock

Introduction

The project is a ROM-based digital lock implemented using hardware design logic and tested in real-time with inputs from users. It mimics an actual safe, where a 6 digit passcode(key from 0-F) is inputted, and the input is compared against the actual passcode. If each key matches, the safe unlocks, but if there's any discrepancy between any of the keys, an alarm goes off.

Final Circuit Diagram



Explanation

The key component of our project was the 27C512 EPROM which allowed us to store a 4 bit key and a 3 bit state at each viable address to compare user's input with data stored in EPROM.
We decided on a digital passcode(123456) and programmed the following data into the EPROM:
0x17
0x11
0x27

0x22
0x37
0x47
0x44
0x57
0x55
0x67
0x66
0x66
0x66
0x66
0x66,

where address bits came from a combination of 3 current state bits and a comparator bit. The output stored the next state and actual key.

We compared input from the hex keypad with the output of EPROM using a 74LS85 comparator. If both were equal, a high comparator bit would go into the ROM and provide the accurate output of the next state.

Next state bits would go into a 74LS273 D-type flip flop which would then output the next state into the address bit of the ROM at each clock pulse(synced with each keypad input).

If a high comparator bit, the next state would be the next state in a finite state machine, else it would go to an alarm state(111), which would then be detected by a state catcher implemented using 7408 ICs.

The bingo(unlock) state also has a state catcher implemented using 7404 and 7408 ICs.

If the alarm state is recognised, it outputs a high bit and a buzzer would go off.

If the bingo state is recognised, it outputs a high bit and an LED light would turn on.

This is checked for each key input, and any wrong key would lead to the alarm going off.

The MR switch on the D-type flip flop allows us to reset the states and input keys from start again(mimicking an override switch for another attempt, or turning off the alarm).

Hardware Implementation Challenges

Our circuit is fairly simple, but that meant that there were many logical errors that we had to account for:

1. Syncing clock of keypad with clock of flip flop IC

Initially, we planned on manually applying a clock pulse using a debounce switch, which would change state after a new key from the keypad was inputted, but that caused multiple syncing issues. The old state would be linked to the old key and new key, causing the alarm to go off. Hence, we decided to directly apply clock output from the keypad to the input of the flip flop, and that resolved the syncing issue.

2. Discrepancies during keypad input
   We changed our keypad three times, firstly, there was no clock output which meant that it did not output the values correctly. Secondly, we figured out that when a new key was being pressed, it would be inputted twice, for example if the first correct key was 1, if we pressed 1, it would output 11, which again caused the alarm to go off because it was the wrong next key. This was a difficult issue to resolve but the LED light on the keypad blinks twice when a key was pressed which indicated that the key was being pressed twice. We changed the keypad again, and the issue was resolved.
3. Data of ROM stored had to be configured precisely
   There was irregularity in the order of data bits we had set out on Proteus initially and the final ROM data. Initially, our ROM input was based like so:
   A3 A2A1A0 where A3 was the output bit from the comparator and A2-A0 was the next state. But when it actually came to configuring the values in the ROM, we miswrote our address order as A3 A2A1A0 where A0 was actually the output bit from the comparator and A3-A1 was the next state, while keeping the order of the ROM data the same as the initial plan, which led to the alarm going off multiple times. This issue was fixed once we realised the order was incorrect.

Improvements

The project was relatively simple to plan but difficult to execute as there were a lot of last minute issues that arose because of incorrect hardware implementation. But this project could be improved on in complexity of design and execution:

1. Use a RAM for writable memory to change/update password in realtime using a microcontroller
2. Add more features like timeout after 3 incorrect inputs or redo a key to reset input using state machine
3. use 7-seg display to show input/progress of user or show number of attempts left

4. use different types of buzzers to indicate lock opened, incorrect password or timeout